



Writing ROS Controllers



Robot Operating System





Lecture 07: Writing ROS Controllers

ROS Controllers & Visualization Plugins

- Modern robotic systems require:
 - **Controllers** for real-time joint/motion control.
 - **Visualization plugins** for robot states, sensor data, and environment perception.
- ROS provides:
 - **ros_control**: standardized controller interfaces.
 - **RViz Plugins**: powerful visualization tools for debugging & interaction.
- This lecture covers the architecture, components, flow, and development steps of both.

What is ROS Control?

- ROS Control is a **framework** (it is a collection of tools, libraries, and conventions that aim to simplify the task of making complex and robust robots) in ROS that **provides standard tools** for **building**, **running**, and **managing robot controllers**.
- This framework includes several **ready-to-use**, **generalized controllers** that are widely used with various robots **Rviz**, **Gazebo**, **CoppeliaSim**, **MoveIt 2**, and **Nav2**.

What is ROS Control?

- It uses **packages** like
 - Controller interfaces
 - Controller managers
 - Transmissions and
 - Hardware interfaces
- ROS Control **works** with both **real robots** and **Gazebo simulation**.

What is ROS Control?

- ROS Control also
 - gives a modular and easy-to-use structure for robot controllers
 - separates controller logic from robot hardware
 - works in real-time for smooth motion control
 - integrates directly with Gazebo through plugins
 - makes testing and development safe and consistent
 - supports many controller types (PID, trajectory, effort, velocity)

Controller Types

- PID
- Trajectory
- Effort
- Velocity

What is PID Controller?

- PID

- PID stands for Proportional–Integral–Derivative, which refers to a common feedback control loop mechanism used to manage and continuously regulate processes.
- A PID controller automatically compares a process's current result to a desired result and makes adjustments to minimize the error.

What is PID Controller?

A PID controller helps a robot reach a target smoothly by correcting errors using three ideas: P, I, and D.

- PID

- **Proportional (P)**: The controller looks at the **current error**. Where the bigger error needs bigger correction. The controller shows fast reaction, but may not be perfect.
- **Integral (I)**: The controller looks at the **past error** and adds corrections from errors over time. It also helps remove remaining small errors.
- **Derivative (D)**: The controller looks at how **fast error** is changing. It predicts future error and reduces overshoot (going beyond a target) and smooths motion.

Question

How does a ROS PID controller work? Draw and explain the control flow.

How PID Controller Works?

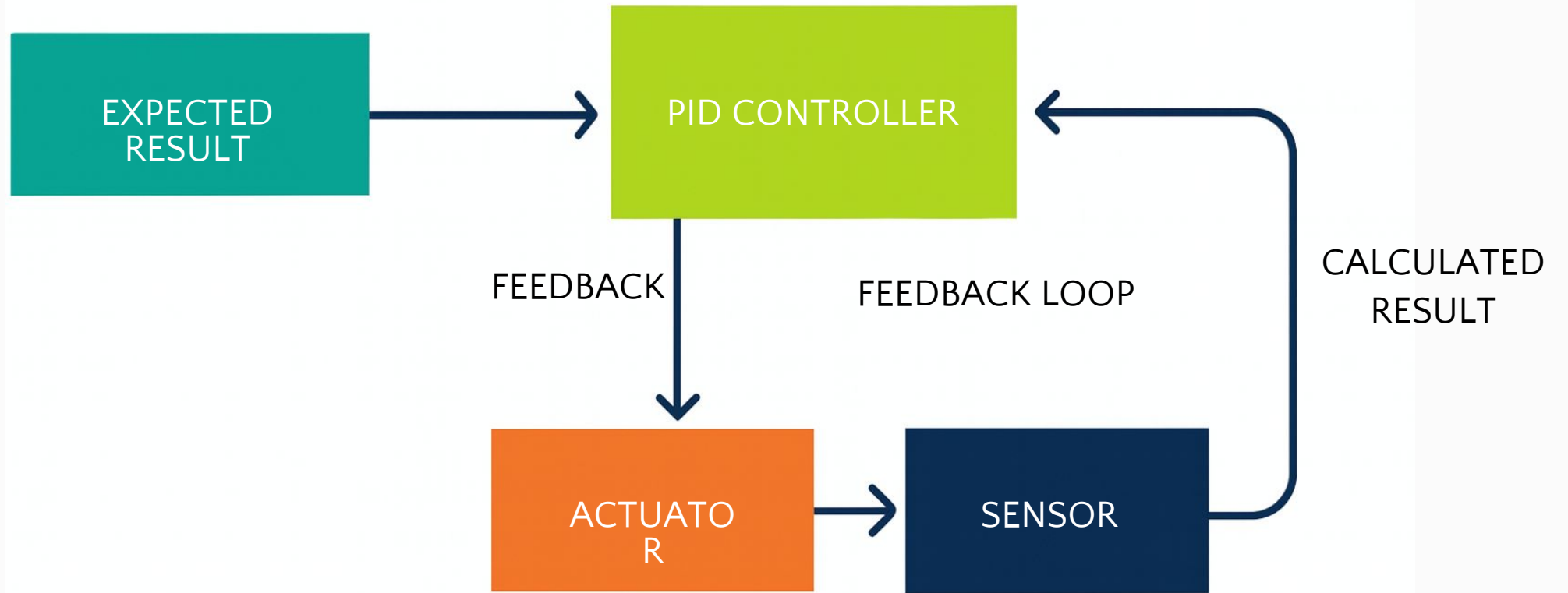


Fig: A control system with PID controller

How PID Controller Works?

$$\text{Error } E(t) = R(t) - Y(t)$$

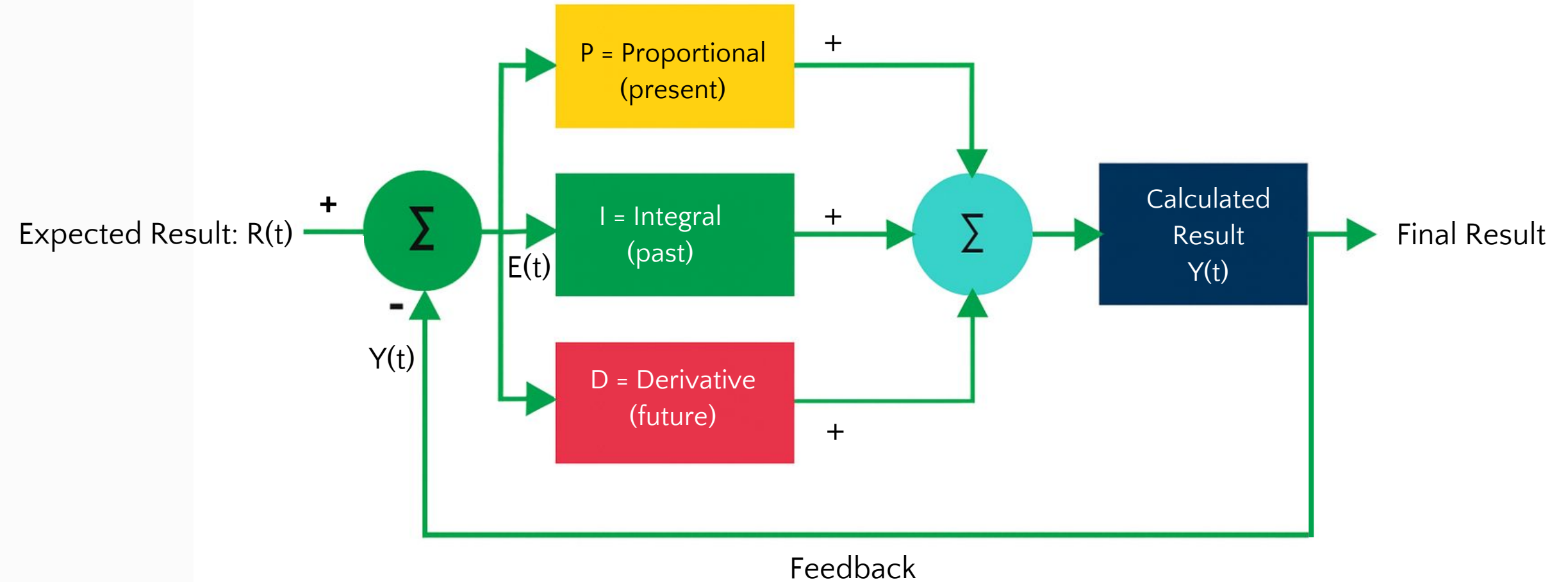


Fig: Block diagram of PID controller

How PID Controller Works?

1. Comparison

- A PID controller starts its operation by comparing the expected result and the resultant value or the actual value, which it get from the sensor devices. Then it generates an error signal which is the difference between the expected result and the resultant value or the actual value.

2. Proportional control

- Proportional (P) term responses to the present error signal and it aims to bring the system closer to the expected result.

How PID Controller Works?

3. Integral control

- The integral (I) part is responsible for addressing any long-term error from evaluating the past errors. And it eliminates steady-state error of a system.

4. Derivative control

- The derivative(D) term observe the rate of change of error, and try to calculate the next error and according to that it makes the system stable.

5. Control signal

- After processing error signal through P, I and D term the controller creates a signal and send it to the actuator to do its job according to that.

What is Trajectory Controller?

- Trajectory Controller

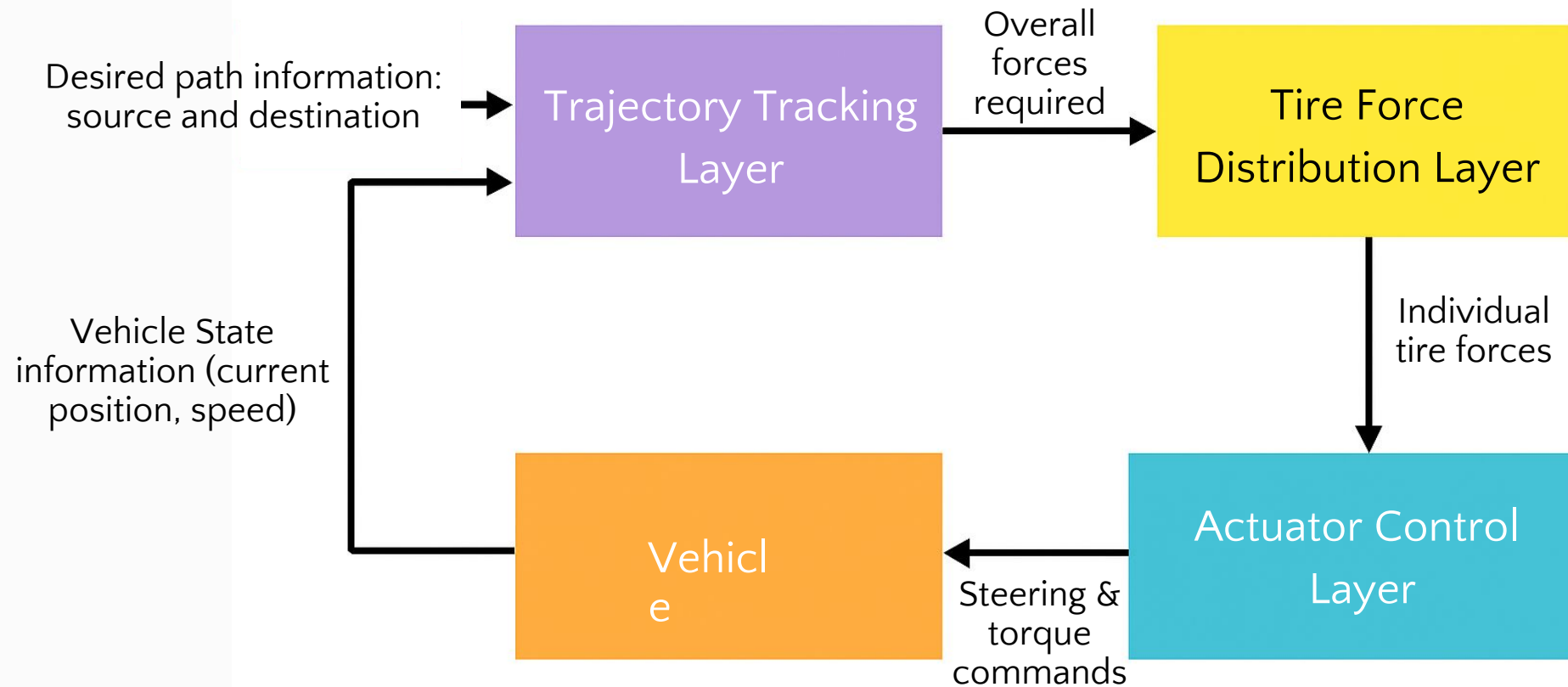
- Trajectory controllers are systems that guide a vehicle or robot along a pre-planned path by using feedback to minimize errors. These controllers are essential for applications like autonomous vehicles, drones, and robotics, where they ensure a system follows a specified trajectory (position, velocity, and acceleration) as accurately as possible, even in the presence of disturbances.

Question

How does a ROS Trajectory Controller work? Draw and explain the control flow.

How Trajectory Controller Works?

- This controller helps a vehicle follow a desired path (from source to destination) by breaking the job into three layers.



How Trajectory Controller Works?

1. Trajectory Tracking Layer (High-Level Controller)

- This layer compares where the vehicle should be vs. where it actually is, and computes a correction.

Input:

- Desired path information (route, waypoints, destination)
- Current vehicle state (position, speed, heading)

Output:

- Required overall forces and moments to stay on the path(e.g., lateral force → steering; longitudinal force → acceleration/braking)

How Trajectory Controller Works?

2. Tire Force Distribution Layer (Mid-Level Controller)

- This layer translates the overall desired force into the specific amount each tire must produce. Because:
 - Each tire has different load
 - Not all tires can produce the same amount of force
 - Modern vehicles use torque vectoring

Example: This layer might decide

- Front-left tire: 120 N
- Front-right tire: 200 N
- Rear-left tire: 140 N
- Rear-right tire: 190 N
- This ensures the vehicle stays stable and avoids tire saturation.

How Trajectory Controller Works?

3. Actuator Control Layer (Low-Level Controller)

- This layer converts all individual forces into real hardware actions, handling:
 - Motor control
 - Brake modulation
 - Steering motor commands
- It also compensates for actuator delays and nonlinearities.

How Trajectory Controller Works?

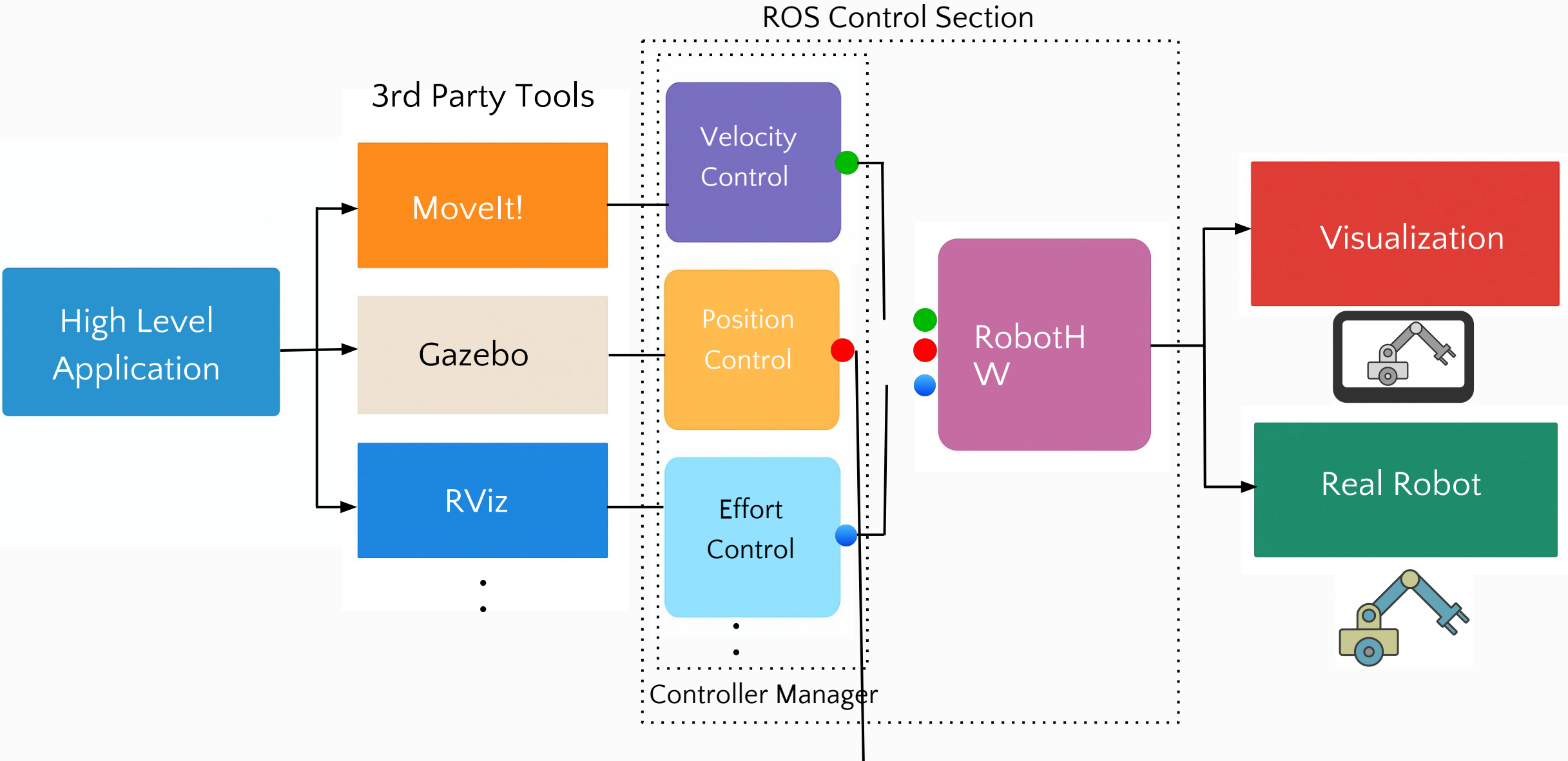
4. Vehicle Layer

- . The vehicle moves according to the commands getting from the Actuator Control Layer and set the new position, velocity and this Vehicle State information (current position, speed) is fed back to the Trajectory Tracking Layer to close the control loop.

Question

How does a ROS controller work? Draw and explain the control flow.

Overview Of ROS Control



High-Level Application (Top Layer)

- This layer defines **what the robot should do**, not how it physically moves.
- It generates **high-level commands** such as **go here**, **pick this**, **follow trajectory**.
- It also contains **user logic** or **custom behaviors**.
- It sends desired motions (e.g., goal positions, velocities) to controllers via third party tools.

3rd Party Tools (Planning & Visualization Layer)

- These tools assist with generating motion or inspecting robot behavior.
- They also produce trajectory or motion commands, which are then passed into ROS Control.

Key Tools

- MoveIt!
 - Performs for inverse kinematics
 - Generates smooth trajectories for arms
- Gazebo
 - Provides full physics simulation
 - Emulates sensors, joints, and actuators
- Rviz
 - Displays robot state and command execution

ros_control Framework (Core Control Layer)

- . This is the heart of the control system.
- . The central component that converts high-level commands into real hardware actions.

a) Controller Manager

- . It is the “operating system” for controllers.
- . It acts as the controller supervisor.
- . Its functionalities
 - . Loads controllers (from plugin system)
 - . Starts/stops controllers
 - . Switches controllers at runtime
 - . Ensures only compatible controllers run together

ros_control Framework (Core Control Layer)

b) Controllers (Compute the actual commands)

- They convert high-level goals into real actuator commands.
- Their functionalities
 - Runs at a real-time control loop (typically 100–1000 Hz)
 - Ensures smooth, stable motion
 - Sends commands downstream to the hardware interface

Controller Type

1. Position Controller: Controls joint position

- Example
 - Move arm to 90°

ros_control Framework (Core Control Layer)

b) Controllers (Compute the actual commands)

2. Velocity Controller: Controls joint speed

- Example
 - Rotate wheel at 1 rad/s

3. Effort Controller: Controls torque/force

- Example
 - Apply 5 Nm torque

RobotHW (Hardware Interface Layer)

- This is an important abstraction layer that connects ROS controllers to actual hardware such as motors and sensors.
- It:
 - `read()`: Reads sensor data (joint positions, velocities, efforts)
 - `write()`: Writes actuator commands (torque, speed, position setpoints)
 - Sends commands (torques, speeds, positions) to actuators
 - Converts the commands to `low-level robot commands` (CAN (Controller Area Network), EtherCAT (Ethernet for Control Automation Technology), Serial)

Real Robot or Simulation (Execution Layer)

- . The final layer that executes commands and physically moves the robot.
- . Two Possible Targets
 1. Simulation
 - . Gazebo / Ignition
 - . Webots
 - . Unity / custom simulators
 2. Real Robot
 - . Industrial robots (UR, Kuka, Franka, etc.)
 - . Mobile robots
 - . Custom robotic arms
- . Therefore, ROS Control behaves identically for both real and simulated robots because only the hardware interface plugin changes.

Question

Write a `custom hardware interface class` in ROS Control by inheriting from `hardware_interface::RobotHW`. Your class should `initialize` the robot, `read` sensor data from hardware, write `actuator` commands, and optionally check for controller conflicts.

Answer

```
#include <hardware_interface/robot_hw.h>

class MyRobot :
    public hardware_interface::RobotHW
{
public:
    MyRobot(); // Setup robot
    MyRobot(ros::NodeHandle &nh, urdf::Model *urdf_model = NULL);

    // Talk to HW
    void read();
    void write();

    // Requirement only if needed ...
    virtual bool checkForConflict(...) const;
}
```

Answer



```
#include <hardware_interface/robot_hw.h>
```

- This line includes the header file for hardware_interface::RobotHW.
- RobotHW is a base class provided by ros_control that you inherit from to define your robot's hardware interface.
- Without this include, the compiler wouldn't know what hardware_interface::RobotHW is.

Answer

```
#include <hardware_interface/robot_hw.h>
```

```
class MyRobot :  
    public hardwareinterface::RobotHW  
{
```

- This declares a new C++ class named MyRobot.
- The : indicates inheritance.
- public hardware_interface::RobotHW means:
 - MyRobot inherits publicly from hardware_interface::RobotHW.
 - So MyRobot is a specialized RobotHW that must implement how to read from and write to your real (or simulated) robot.

Answer



```
#include <hardware_interface/robot_hw.h>

class MyRobot :
    public hardwareinterface::RobotHW
{
    public :
```

- This is an access specifier.
- Everything after public: is accessible from outside the class (e.g., other code can call MyRobot::read()).
- Typical structure: public:, protected:, private:
- Here, only public: is used.

Answer



```
#include <hardware_interface/robot_hw.h>

class MyRobot :
    public hardwareinterface::RobotHW
{
    MyRobot(); // Setup robot
```

- This is a default constructor declaration.
 - Initialize internal variables
 - Set up communication (e.g., CAN, serial, etc.)
 - Prepare the robot for use

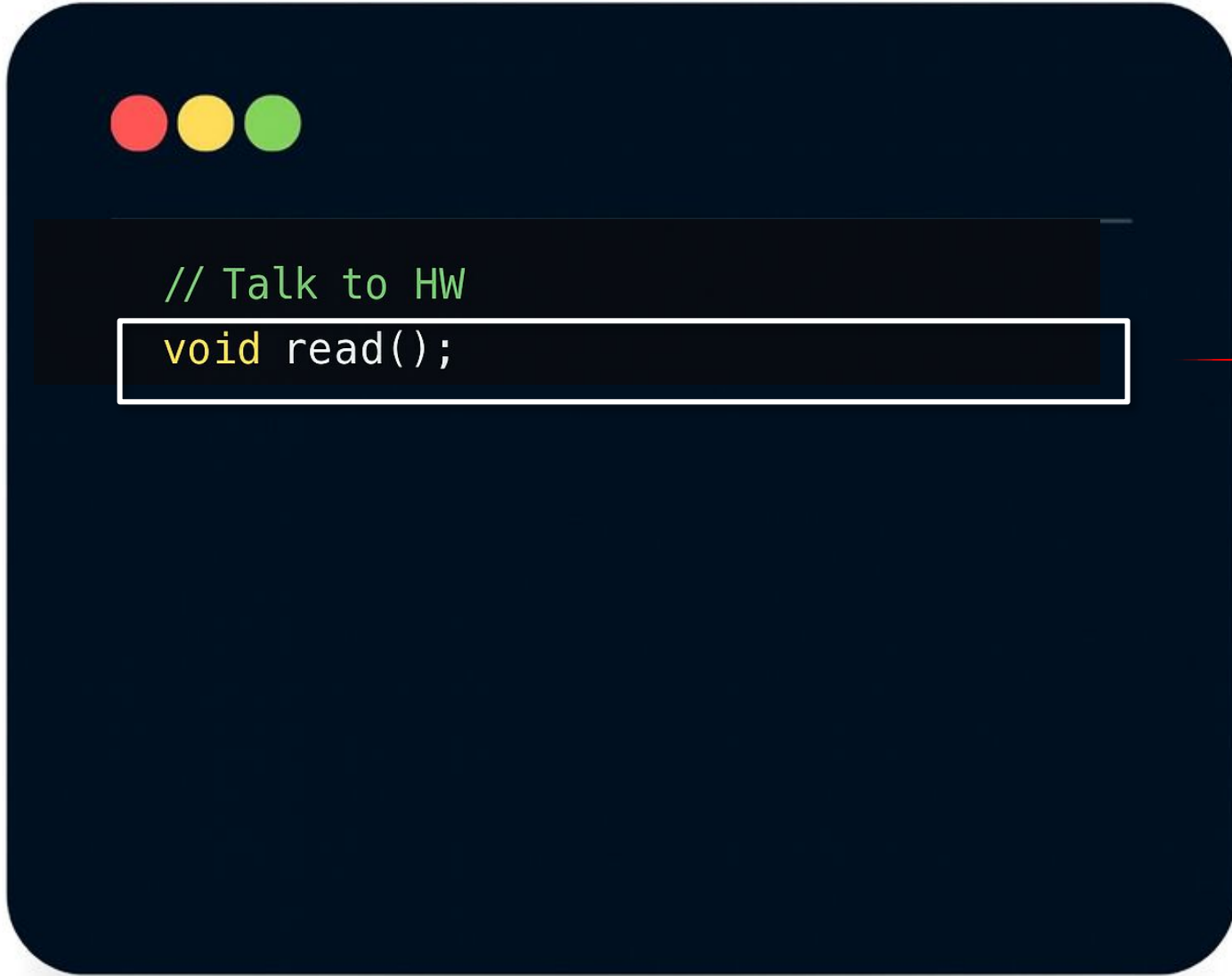
Answer

```
#include <hardware_interface/robot_hw.h>

class MyRobot :
    public hardwareinterface::RobotHW
{
    MyRobot(); // Setup robot
    MyRobot(ros::NodeHandle &nh,
            urdf::Model *urdf_model = NULL);
```

- This is an overloaded constructor (another version of MyRobot with parameters).
 - Loads robot configuration from the parameter server
 - Reads URDF
 - Registers joints and interfaces with RobotHW.

Answer



```
// Talk to HW
```

```
void read();
```

- Read the current state from the hardware:
 - Joint positions
 - Joint velocities
 - Joint efforts

Answer



```
// Talk to HW  
void read();  
void write();
```

- 
- Write commands from ROS controllers to the hardware:
 - Position commands
 - Velocity commands
 - Effort/torque commands

Answer



```
// Talk to HW
void read();
void write();

// Requirement only if needed ...
virtual bool checkForConflict(...) const;
}
```

- Used when loading or switching controllers.
- Checks whether two controllers are conflicting (e.g., both trying to control the same joint in incompatible ways).
- Returns:
 - true if there is a conflict
 - false if it is safe to proceed

Q & A





Thank
you!